LAB 5: STATIC CODE ANALYSIS

Name: Adarsh Prakash | SRN: PES2UG24CS027 | Section: A

1. Objective

To evaluate the school transport register Python program using Pylint, Bandit, and Flake8; identify code-quality, security, and style issues; apply appropriate fixes; and compare the before-and-after reports.

2. Tools and Environment

Environment: Windows PowerShell, Python 3.12.10, and a project virtual environment (.venv). Installed tool versions recorded during the lab: Pylint 4.1.3, Bandit 1.9.4, and Flake8 7.4.1. Assigned program: *school_transport.py*.

3. Issues Identified and Fixes

Issue	Type / Tool	Before finding	Fix applied	Verification / result
SQL string interpolation	Security / Bandit	B608: possible SQL injection from an f-string INSERT statement (Medium severity, Medium confidence).	Replaced interpolated SQL values with a parameterized query using placeholders and a tuple of values.	AFTER: Bandit reported no issues identified.
Shell execution	Security / Bandit	B602: subprocess.run used shell=True (High severity, High confidence); subprocess import also flagged as B404.	Removed subprocess shell execution and used a normal Python print statement for the dispatch alert.	AFTER: no Bandit findings.
Runtime assert	Security / Bandit; correctness	B101: assert used for a runtime fee-total check.	Replaced assert with an explicit conditional that raises RuntimeError if the expected total is not met.	Program execution verified; the R1 fee check passes.
Bare exception handling	Quality / Pylint and Flake8	Bare except clause could hide unrelated errors; Flake8 E722 and Pylint warning.	Caught sqlite3.Error explicitly and returned the fallback value for database errors.	AFTER: no Flake8 findings; Pylint score 10.00/10.
Missing docstrings and naming conventions	Quality / Pylint	Missing module/function docstrings and camelCase function names.	Added a module docstring and function docstrings; renamed functions to snake_case and updated call sites.	AFTER: Pylint score 10.00/10.
File handling and encoding	Quality / Pylint	File opened without explicit encoding and not protected by a context manager.	Used a with statement and encoding="utf-8" so the file is closed reliably.	Roster export works and writes the expected names.
Line length and whitespace	Style / Flake8 and Pylint	Long lines, missing whitespace after commas, and blank-line formatting violations.	Wrapped long expressions and corrected spacing and function separation.	AFTER: Flake8 output is empty (0 reported violations).
Unused parameter	Quality / Pylint	The driver parameter in route_roster was unused.	Removed the unused parameter and updated the function call.	AFTER: Pylint score 10.00/10.

4. Analyzer Results Summary

Tool	Before	After
Pylint	Score: 4.00/10; multiple quality/style findings.	Score: 10.00/10.
Bandit	4 findings: 1 High (B602), 1 Medium (B608), and 2 Low (B404 and B101).	No issues identified; 0 Low, 0 Medium, 0 High.
Flake8	9 findings, including line-length, whitespace, blank-line, and bare-except issues.	No output; 0 reported violations.

Functional verification: running school_transport.py printed “dispatching bus R1” and “['Zoya']”. The R1 fee total check passed, and the exported route_r1.txt roster contains Tara and Kabir.

Commit history recorded in the project repository: (1) Improve Python code quality for Pylint; (2) Fix security findings identified by Bandit; (3) Add static analysis reports and lab documentation.

5. Reflection

Q1. What is static code analysis, and how is it useful?

Static code analysis checks source code without executing it. It can identify potential defects, unsafe coding patterns, and style problems early. It complements rather than replaces tests.

Q2. What differences did you observe between Pylint, Bandit, and Flake8?

Pylint examines broad code quality, naming, documentation, and possible design issues, and gives a score. Bandit focuses on security-sensitive Python patterns. Flake8 reports style and basic code issues such as line length, whitespace, and bare exception handling.

Q3. Which issues were most important, and why?

The SQL interpolation and shell=True findings were important because they can expose a program to unsafe input handling or unintended shell commands. Replacing assert with an explicit runtime check and narrowing exception handling also made behavior more predictable.

Q4. How did static analysis improve the program, and what are its limitations?

The tools helped make database writes safer, remove shell execution, improve exception handling and file management, and standardize naming and formatting. A clean report does not prove complete correctness or security; tests and manual review are still needed.

6. Conclusion

The school transport program was updated to address the identified security, quality, and style findings. Final checks achieved a Pylint score of 10.00/10, no Bandit findings, and no Flake8 violations. The before-and-after reports and issue documentation are included with the project.